

[页面](#) / ... / [kafka使用过程总结](#)

Flink kafka自定义时间范围

由 朱明创建, 最后修改于六月 30, 2022 [🔗](#)52 views since 29 Jun 2022

一、背景

目前flink的kafka consumer目前支持的自定义的接口只有开始的时间戳和offset，其中并没有范围的选项。

```
setStartFromTimestamp(long startupOffsetsTimestamp)
```

```
setStartFromSpecificOffsets(Map<KafkaTopicPartition, Long> specificStartupOffsets)
```

通过分析FlinkKafkaConsumer源码，消费者启动是由FlinkKafkaConsumerBase.run实现，通过创建一个KafkaFetcher，由KafkaFetcher启动消费kafka topic的线程，并获取数据。

关于kafka数据选择消费的的实现逻辑在KafkaFetcher.runFetchLoop()循环体中。所以在循环体中加入停止时间即可。

二、实现

关键是runFetchLoop()方法，增加一个循环结束的条件，当拉取数据到达指定时间戳时，停止拉取，关闭线程。

首先是构建CustomFlinkKafkaConsumer代替FlinkKafkaConsumer，加入stopConsumingTimestamp时间戳，和自定义的54行setIntervalFromTimestamp方法。

在setIntervalFromTimestamp方法中调用setStartFromTimestamp官方提供的接口设置起始时间。

AbstractFetcher方法需要返回自定义的KafkaFetcher.runFetchLoop()，这样kafka会调用我们自定义实现的runFetchLoop ()

```
1
2 package org.apache.flink.streaming.connectors.kafka;
3
4 import com.mediarx.realtimeexchangeduplicator.domain.RawLog;
5 import org.apache.flink.api.common.eventtime.WatermarkStrategy;
6 import org.apache.flink.api.common.serialization.DeserializationSchema;
7 import org.apache.flink.metrics.MetricGroup;
8 import org.apache.flink.streaming.api.context.StreamingRuntimeContext;
9 import org.apache.flink.streaming.connectors.kafka.config.OffsetCommitMode;
10 import org.apache.flink.streaming.connectors.kafka.internals.CustomKafkaFetcher;
11 import org.apache.flink.streaming.connectors.kafka.internals.AbstractFetcher;
12 import org.apache.flink.streaming.connectors.kafka.internals.KafkaTopicPartition;
13 import org.apache.flink.util.SerializedValue;
14
15 import java.util.List;
16 import java.util.Map;
17 import java.util.Properties;
18 import java.util.regex.Pattern;
19
20 /**
21  * @author zhuming
22  * @date 2022/6/23 17:14
23  */
24 public class CustomFlinkKafkaConsumer extends FlinkKafkaConsumer<RawLog> {
25     private long stopConsumingTimestamp;
26
27     public CustomFlinkKafkaConsumer(String topic, DeserializationSchema<RawLog> valueDeserializer, Properties props) {
28         super(topic, valueDeserializer, props);
29     }
30
31     public CustomFlinkKafkaConsumer(String topic, KafkaDeserializationSchema<RawLog> deserializer, Properties props) {
32         super(topic, deserializer, props);
33     }
34
35     public CustomFlinkKafkaConsumer(List<String> topics, DeserializationSchema<RawLog> deserializer, Properties props) {
36         super(topics, deserializer, props);
37     }
38
39     public CustomFlinkKafkaConsumer(List<String> topics, KafkaDeserializationSchema<RawLog> deserializer, Properties props) {
40         super(topics, deserializer, props);
41     }
42
43     public CustomFlinkKafkaConsumer(Pattern subscriptionPattern, DeserializationSchema<RawLog> valueDeserializer, Properties props) {
44         super(subscriptionPattern, valueDeserializer, props);
45     }
46
47     public CustomFlinkKafkaConsumer(Pattern subscriptionPattern, KafkaDeserializationSchema<RawLog> deserializer, Properties props) {
48         super(subscriptionPattern, deserializer, props);
49     }
50
51     private void setStopConsumingTimestamp(long stopConsumingTimestamp) {
52         this.stopConsumingTimestamp = stopConsumingTimestamp;
53     }
54     //指定时间范围
55     public FlinkKafkaConsumerBase<RawLog> setIntervalFromTimestamp(long startupOffsetsTimestamp, long stopConsumingTimestamp) {
56         setStartFromTimestamp(startupOffsetsTimestamp);
57         setStopConsumingTimestamp(stopConsumingTimestamp);
58     }
59 }
```

```

57         if (startupOffsetsTimestamp > stopConsumingTimestamp) {
58             throw new IllegalArgumentException("The start consuming time " + startupOffsetsTimestamp + "exceeds the end
59         } else {
60             return super.setStartFromTimestamp(startupOffsetsTimestamp);
61         }
62     }
63
64     @Override
65     protected AbstractFetcher<RawLog, ?> createFetcher(SourceContext<RawLog> sourceContext, Map<KafkaTopicPartition, Lon
66         //自定义KafkaFetcher, 主要传入stopConsumingTimestamp时间戳, 在循环消费体中加入判断
67         return new CustomKafkaFetcher(sourceContext, assignedPartitionsWithInitialOffsets, watermarkStrategy, runtimeCon
68     }
69 }

```

自定义CustomKafkaFetcher 主要重构了 runFetchLoop函数, 使得在到达时间后停止消费。

保证消费完退出, 清晰查看job运行状态边界, 并查看flink的各slot消费边界情况

就是加了75行的代码, if (record.timestamp()) > stopConsumingTimestamp) { this.running = false; break; }

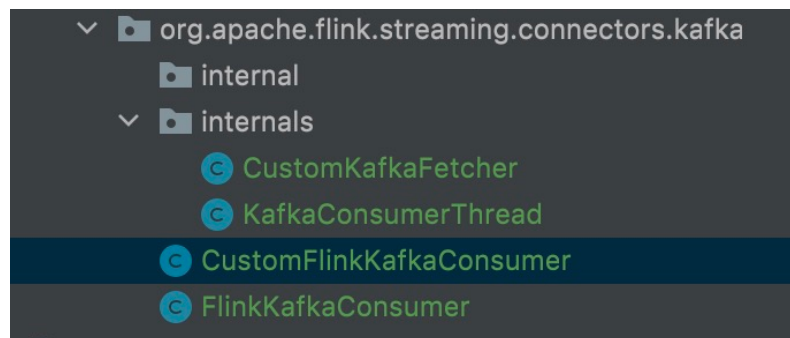
主要参考上面流的代码实现了时间戳结束的判断。

注: 在结束后设置单个slot状态为结束finished, 此时flink的cp在UI里面不再展示, 但实际其它slot的cp都正常运行。

取消offset提交, 保证不影响该consumer的正常offset

第123行的 doCommitInternalOffsetsToKafka 中的注释代码为源码, 因为测试的时候不需要提交offset, 所以置为空。如果消费完这些数据后, 需要从结束时间戳消费, 那么需要取消注释, 提交offset。

注: 因为该方法中 this.consumerThread.setOffsetsToCommit需要调用flink官方包下的protected方法, 所以引入该方法需要保证自己的包路径和官方的包路径一致, 也就是kafka-connector包, 具体写法如下。



```

1 package org.apache.flink.streaming.connectors.kafka.internals;
2
3
4 import com.mediarv.realtimeexchangeduplicator.domain.RawLog;
5 import org.apache.flink.annotation.Internal;
6 import org.apache.flink.api.common.eventtime.WatermarkStrategy;
7 import org.apache.flink.kafka.shaded.org.apache.kafka.clients.consumer.ConsumerRecord;
8 import org.apache.flink.kafka.shaded.org.apache.kafka.clients.consumer.ConsumerRecords;
9 import org.apache.flink.kafka.shaded.org.apache.kafka.clients.consumer.OffsetAndMetadata;
10 import org.apache.flink.kafka.shaded.org.apache.kafka.common.TopicPartition;
11 import org.apache.flink.metrics.MetricGroup;
12 import org.apache.flink.streaming.api.functions.source.SourceFunction;
13 import org.apache.flink.streaming.connectors.kafka.KafkaDeserializationSchema;
14 import org.apache.flink.streaming.runtime.tasks.ProcessingTimeService;
15 import org.apache.flink.util.Preconditions;
16 import org.apache.flink.util.SerializedValue;
17 import org.slf4j.Logger;
18 import org.slf4j.LoggerFactory;
19
20 import javax.annotation.Nonnull;
21 import java.util.*;
22 import java.util.concurrent.ConcurrentLinkedQueue;
23
24 /**
25  * @author zhuming
26  * @date 2022/6/23 17:24
27  */
28
29 @Internal
30 public class CustomKafkaFetcher extends AbstractFetcher<RawLog, TopicPartition> {
31     private static final Logger LOG = LoggerFactory.getLogger(CustomKafkaFetcher.class);
32     private final KafkaDeserializationSchema<RawLog> deserializer;

```

```

32     private final Handover handover;
33     private final KafkaConsumerThread<RawLog> consumerThread;
34     private volatile boolean running = true;
35
36     public long stopConsumingTimestamp;
37
38
39     public CustomKafkaFetcher(SourceFunction.SourceContext<RawLog> sourceContext, Map<KafkaTopicPartition, Long> assign
40         super(sourceContext, assignedPartitionsWithInitialOffsets, watermarkStrategy, processingTimeProvider, autoWater
41         this.deserializer = deserializer;
42         this.handover = new Handover();
43         this.consumerThread = new KafkaConsumerThread<>(LOG, this.handover, kafkaProperties, this.unassignedPartitionsQ
44         this.stopConsumingTimestamp = stopConsumingTimestamp;
45     }
46
47     public void runFetchLoop() throws Exception {
48         try {
49             final Handover handover = this.handover;
50
51             // kick off the actual Kafka consumer
52             consumerThread.start();
53
54             while (running) {
55                 // this blocks until we get the next records
56                 // it automatically re-throws exceptions encountered in the consumer thread
57                 final ConsumerRecords<byte[], byte[]> records = handover.pollNext();
58
59                 // get the records for each topic partition
60                 for (KafkaTopicPartitionState<RawLog, TopicPartition> partition : subscribedPartitionStates()) {
61
62                     List<ConsumerRecord<byte[], byte[]>> partitionRecords =
63                         records.records(partition.getKafkaPartitionHandle());
64
65                     for (ConsumerRecord<byte[], byte[]> record : partitionRecords) {
66                         final RawLog value = deserializer.deserialize(record);
67
68                         if (deserializer.isEndOfStream(value)) {
69                             // end of stream signaled
70                             running = false;
71                             break;
72                         }
73
74                         //stop fetching and emitting
75                         if (record.timestamp() > stopConsumingTimestamp) {
76                             this.running = false;
77                             break;
78                         }
79
80                         // emit the actual record. this also updates offset state atomically
81                         // and deals with timestamps and watermark generation
82                         ConcurrentLinkedQueue<RawLog> queue = new ConcurrentLinkedQueue<>();
83                         queue.add(value);
84                         emitRecord(queue, partition, record.offset(), record);
85                     }
86                 }
87             }
88         } finally {
89             // this signals the consumer thread that no more work is to be done
90             consumerThread.shutdown();
91         }
92
93         // on a clean exit, wait for the runner thread
94         try {
95             consumerThread.join();
96         } catch (InterruptedException e) {
97             // may be the result of a wake-up interruption after an exception.
98             // we ignore this here and only restore the interruption state
99             Thread.currentThread().interrupt();
100        }
101    }
102
103     protected void emitRecord(Queue<RawLog> records, KafkaTopicPartitionState<RawLog, TopicPartition> partition, long o
104         this.emitRecordsWithTimestamps(records, partition, offset, consumerRecord.timestamp());
105
106     }
107
108     public void cancel() {
109         this.running = false;

```

```

110         this.handover.close();
111         this.consumerThread.shutdown();
112     }
113
114
115     protected String getFetcherName() {
116         return "Kafka Fetcher";
117     }
118
119     public TopicPartition createKafkaPartitionHandle(KafkaTopicPartition partition) {
120         return new TopicPartition(partition.getTopic(), partition.getPartition());
121     }
122
123     public void doCommitInternalOffsetsToKafka(Map<KafkaTopicPartition, Long> offsets, @Nonnull KafkaCommitCallback com
124     //         List<KafkaTopicPartitionState<RawLog, TopicPartition>> partitions = this.subscribedPartitionStates();
125     //         Map<TopicPartition, OffsetAndMetadata> offsetsToCommit = new HashMap(partitions.size());
126     //         Iterator var5 = partitions.iterator();
127     //
128     //         while(var5.hasNext()) {
129     //             KafkaTopicPartitionState<RawLog, TopicPartition> partition = (KafkaTopicPartitionState)var5.next();
130     //             Long lastProcessedOffset = (Long)offsets.get(partition.getKafkaTopicPartition());
131     //             if (lastProcessedOffset != null) {
132     //                 Preconditions.checkState(lastProcessedOffset >= 0L, "Illegal offset value to commit");
133     //                 long offsetToCommit = lastProcessedOffset + 1L;
134     //                 offsetsToCommit.put(partition.getKafkaPartitionHandle(), new OffsetAndMetadata(offsetToCommit));
135     //                 partition.setCommittedOffset(offsetToCommit);
136     //             }
137     //         }
138     //         try {
139     //             this.consumerThread.setOffsetsToCommit(offsetsToCommit, commitCallback);
140     //         } catch (Exception e){
141     //             LOG.info(e.getLocalizedMessage());
142     //         }
143     }
144 }

```

scala中的consumer的创建代码如下

```

def createKafkaConsumer(appProp: Properties, kafkaConsumerProp: Properties): FlinkKafkaConsumer[RawLog] = {
    val topics = Objects.requireNonNull(appProp.getProperty("topics"), message = "Topics must be set").split(",")
    val consumer = new CustomFlinkKafkaConsumer(topics, new RawLogDeserializer, kafkaConsumerProp)
    consumer.setIntervalFromTimestamp( startupOffsetsTimestamp = 1656374400000L, stopConsumingTimestamp = 1656385200000L)
    consumer
}
// val consumer = new FlinkKafkaConsumer[RawLog](topics, new RawLogDeserializer, kafkaConsumerProp)
// consumer.setStartFromTimestamp(1656374400000L)
// consumer
}
// no using cache as window, just single input and single output

```

无标签